

Universitat Autònoma de Barcelona

Master Degree in Modelling for Science and Engineering

Institution / Lab: IIIA-CSIC

Technical Memory Draft Baseline
End-to-End Planner Dialogue Flow

Formal architecture and contract section for technical review.

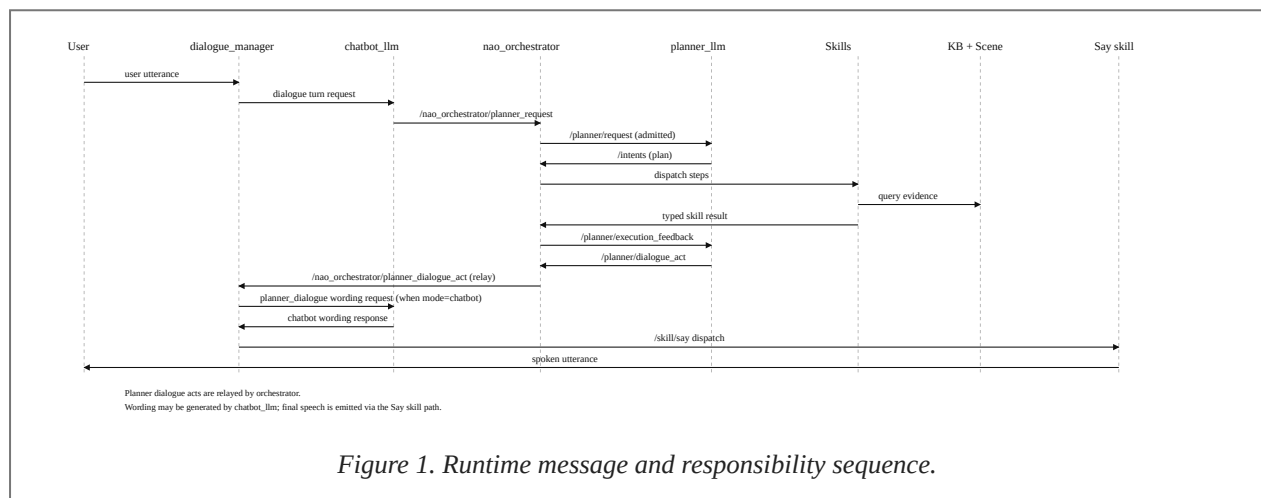
Document type:	Architecture and contract walkthrough
Academic year:	2025-2026
Date:	2026-06-01
Scope:	Runtime flow, JSON contracts, replanning lineage, and dialogue relay behavior.
Repository:	https://github.com/ieverythng/nao-ros4hri-bridge

End-to-End User to Planner Dialogue Flow

This document delivers a full contract walkthrough of the active planner stack, from user intent ingress to execution feedback and dialogue-act emission. The emphasis is strict payload transparency: each tracked seam includes concrete JSON structures with field-level definitions.

Its objective is to make planning, execution, and dialogue responsibilities auditable across node boundaries while preserving deterministic behavior, clean ownership, and traceable runtime decisions.

1) Runtime Sequence Overview



2) Contract Coverage Matrix

The matrix below summarizes the stable interfaces used by the runtime stack. It is intended as a quick reference before reading each contract in detail.

Contract	Channel	Producer	Consumer
Chatbot turn output	internal turn result JSON	chatbot_llm	chatbot handoff path
Planner request envelope	/nao_orchestrator/planner_request, /planner/request (Intent)	chatbot_llm, nao_orchestrator	nao_orchestrator, planner_llm
Knowledge snapshot	grounded_context.knowledge_snapshot	chatbot_llm	planner_llm
Scene summary	grounded_context.scene_summary	chatbot_llm	planner_llm

Contract	Channel	Producer	Consumer
Planner request	Intent.data on /planner/request	chatbot_llm, nao_orchestrator	planner_llm
Planner output	Intent.data on /intents	planner_llm	nao_orchestrator
Execution feedback	/planner/execution_feedback	nao_orchestrator	planner_llm
Planner dialogue act	/planner/dialogue_act	planner_llm	nao_orchestrator and dialogue_manager relay

3) Contract A: chatbot_llm Turn Output

This contract captures intent-routing output before planner admission. It defines the semantic request frame that downstream components normalize into planner transport payloads.

```
{
  "verbal_ack": "Okay, I will do that.",
  "route": "execution",
  "confidence": 0.82,
  "user_intent": {
    "type": "look_at",
    "goal": "look at the person",
    "goal_text": "look at the person",
    "scene_targets": ["person"],
    "request_kind": "new_goal",
    "interaction_mode": "speech"
  }
}
```

Variable	Description
<code>verbal_ack</code>	Optional acknowledgement sentence.
<code>route</code>	Routing value: dialogue, knowledge_query, or execution.
<code>confidence</code>	Normalized confidence in [0.0, 1.0].
<code>user_intent</code>	Intent frame used to construct planner request payload.

4) Contract B: Planner Request Transport Envelope (Intent)

This envelope is the ROS transport seam between dialogue-originated requests and planner ingestion. It separates transport metadata from structured planner request content.

```
{
  "intent": "planner_request",
  "source": "user_123",
  "modality": "speech",
  "confidence": 0.82,
  "priority": 128,
  "data": {
    "request_id": "turn_123",
    "goal_id": "goal_turn_123",
    "request_kind": "new_goal",
    "goal_text": "look at the person",
    "normalized_intents": ["look_at"],
    "scene_targets": ["person"],
    "dialogue_context": [],
    "requested_plan": [],
    "grounded_context": {
      "knowledge_snapshot": {},
      "scene_summary": {}
    },
    "planner_mode": "default",
    "interaction_mode": "speech",
    "dialogue_turn_id": "role:turn"
  }
}
```

Variable	Description
intent	Planner ingress label; expected value is <code>planner_request</code> .
source, modality	Transport metadata from upstream dialogue/agent context.
confidence	Planner handoff confidence (bounded to [0.0, 1.0]).
priority	Planner request priority used at Intent transport layer (default 128 in adapter).
data	Canonical planner request payload consumed by <code>planner_llm</code> .

5) Contract C: Knowledge Snapshot

The knowledge snapshot is a compact symbolic grounding seam. It gives the planner normalized identifiers and classes without exposing full graph payloads.

```
{
  "schema_version": "knowledge_snapshot_v2",
  "captured_at_sec": 1777040000.0,
  "references": [
    {"normalized_name": "person", "id": "person_1", "type": "Person"},
  ]
}
```

```
    {"normalized_name": "cup", "id": "cup_1", "type": "Cup"}
  ],
  "counts": {"entities": 2, "people": 1, "objects": 1}
}
```

Variable	Description
schema_version	Contract-version guard for downstream compatibility.
captured_at_sec	Timestamp indicating when snapshot references were captured.
references[]	Compact symbolic entities with normalized_name, id, and type.
counts	Aggregated entity totals used for quick planning checks.

6) Contract D: Scene Summary

The scene summary carries observation-driven evidence from perception backends. It is intentionally bounded and typed to keep planner context interpretable and stable.

```
{
  "schema_version": "scene_summary_v2",
  "observer": "myself",
  "backend": "emorobcare_cv",
  "captured_at_sec": 1777040000.0,
  "objects": [
    {
      "entity_id": "detected_cup_320_240",
      "label": "cup",
      "kb_class": "Cup",
      "score": 0.91,
      "tracker_id": "",
      "source": "emorobcare_cv",
      "center_x": 320.0,
      "center_y": 240.0,
      "last_seen_sec": 1777040000.0
    }
  ],
  "people": [
    {
      "id": "person_1",
      "label": "person",
      "type": "Person",
      "source": "emorobcare_cv",
      "score": 0.88,
      "center_x": 210.0,
      "center_y": 180.0,
      "last_seen_sec": 1777040000.0
    }
  ]
}
```

Variable	Description
<code>observer</code> , <code>backend</code>	Provenance of scene extraction (who observed and which detection backend).
<code>captured_at_sec</code>	Timestamp of this perception snapshot.
<code>objects[]</code>	Object detections with labels, class, score, image position, and recency.
<code>people[]</code>	Person detections with identity fields and confidence metadata.

7) Contract E: Planner Request (Intent.data)

This is the canonical request body consumed by `planner_llm`. It consolidates intent semantics, dialogue continuity, and bounded grounding references.

```
{
  "request_id": "turn_123",
  "goal_id": "goal_turn_123",
  "parent_goal_id": "",
  "supersedes_goal_id": "",
  "request_kind": "new_goal",
  "goal_text": "look at the person and report completion",
  "normalized_intents": ["look_at"],
  "scene_targets": ["person"],
  "dialogue_context": ["assistant:Okay, I will do that."],
  "requested_plan": [],
  "grounded_context": {
    "knowledge_snapshot": {
      "schema_version": "knowledge_snapshot_v2",
      "captured_at_sec": 1777040000.0,
      "references": [{"normalized_name": "person", "id": "person_1", "type": "Person"},
      "counts": {"entities": 1, "people": 1, "objects": 0}
    },
    "scene_summary": {
      "schema_version": "scene_summary_v2",
      "observer": "myself",
      "backend": "emorobcare_cv",
      "captured_at_sec": 1777040000.0,
      "objects": [],
      "people": [{"id": "person_1", "label": "person", "type": "Person", "source": "myself"}]
    }
  },
  "planner_mode": "default",
  "interaction_mode": "speech",
  "dialogue_turn_id": "role:turn"
}
```

Variable	Description
<code>request_id</code>	Per-turn identifier used for traceability and idempotence.
<code>goal_id</code>	Goal continuity key across updates/replans.
<code>parent_goal_id</code> , <code>supersedes_goal_id</code>	Optional lineage and replacement linkage between goals.
<code>request_kind</code>	Normalized transition kind: <code>new_goal</code> , <code>goal_update</code> , <code>clarification_answer</code> , or <code>cancel_request</code> .
<code>goal_text</code>	Planner-facing objective statement.
<code>normalized_intents</code> , <code>scene_targets</code>	Intent and target abstraction used for plan synthesis.
<code>dialogue_context</code>	Bounded turn history slice that helps continuity for clarifications.
<code>requested_plan</code>	Optional planner hint; normalized as plan-step objects when present.
<code>grounded_context</code>	Grounding envelope composed of <code>knowledge_snapshot</code> and <code>scene_summary</code> .
<code>planner_mode</code>	Policy profile selector (for example <code>default</code> , <code>multi_step</code>).
<code>interaction_mode</code> , <code>dialogue_turn_id</code>	Dialogue-channel metadata and turn correlation id.

Contract guardrail. Legacy fields such as `goal_token`, `world_model_snapshot`, and `world_model_text` are intentionally excluded from the active seam.

8) Contract F: Planner Output (Intent.data on /intents)

Planner output defines executable step policy and communication behavior. It is the primary upstream artifact consumed by the orchestrator execution loop.

```
{
  "grounded_context": {
    "knowledge_snapshot": {},
    "scene_summary": {}
  },
  "plan": {
```

```

"goal_id": "goal_turn_123",
"plan_id": "plan_123",
"plan_version": 2,
"status": "planning",
"validation_status": "draft",
"failure_reason": "",
"user_facing_reason": "",
"replan_hint": "",
"retry_budget": 1,
"scene_targets": ["person"],
"communication_policy": {
  "emit_acknowledge": false,
  "emit_progress": false,
  "emit_completion": true,
  "emit_failure": true
},
"communication_policy_source": "planner_engine:plan",
"steps": [
  {
    "id": "step_1",
    "type": "look_at",
    "name": "look_at",
    "args": {"target_frame": "person_1"},
    "requires": [],
    "on_failure": "replan",
    "retry_budget": 0
  }
]
}
}

```

Variable	Description
<code>grounded_context</code>	Normalized T0 envelope currently echoed in planner output for traceability.
<code>plan.goal_id</code> , <code>plan.plan_id</code> , <code>plan.plan_version</code>	Lineage tuple for join decisions and duplicate suppression.
<code>status</code> , <code>validation_status</code>	Planner lifecycle and contract validity status.
<code>failure_reason</code> , <code>user_facing_reason</code> , <code>replan_hint</code>	Failure semantics and follow-up guidance.
<code>retry_budget</code>	Plan-level remaining retries for replanning control.
<code>communication_policy</code>	Boolean flags controlling acknowledge/progress/completion/failure relays.
<code>communication_policy_source</code>	Source tag for how communication policy was selected.

Variable	Description
<code>steps[]</code>	Executable sequence with stable <code>id</code> , normalized <code>type</code> , args, dependencies, and failure policy.

9) Contract G: Execution Feedback (/planner/execution_feedback)

Execution feedback is the typed return seam from orchestrator to planner supervision. It enables deterministic replanning, clarification decisions, and trace-quality diagnostics.

```
{
  "goal_id": "goal_turn_123",
  "plan_id": "plan_123",
  "plan_version": 2,
  "intent": "raw_user_input",
  "source": "planner_llm",
  "event_type": "step_succeeded",
  "status": "succeeded",
  "reason": "",
  "validation_status": "valid",
  "replan_hint": "",
  "retry_budget": 1,
  "blocking": false,
  "unmet_preconditions": [],
  "needs_user_input": false,
  "scene_targets": ["person"],
  "validation_errors": [],
  "timestamp_sec": 1777040000.0,
  "result_summary": "Look-at completed for person_1",
  "result_payload": {
    "skill": "look_at",
    "target": "person_1",
    "target_kind": "people",
    "target_found": true,
    "summary_text": "Look-at completed for person_1",
    "confidence_policy": "grounded_current_observation"
  },
  "step": {
    "id": "step_1",
    "type": "look_at",
    "name": "look_at",
    "retry_budget": 0,
    "on_failure": "replan",
    "requires": []
  }
}
```

Variable	Description
<code>goal_id</code> , <code>plan_id</code> , <code>plan_version</code>	Plan lineage identifiers consumed by the replanning loop.
<code>intent</code> , <code>source</code>	Producer context for trace diagnostics.
<code>event_type</code>	Normalized event class (for example <code>plan_accepted</code> , <code>step_started</code> , <code>step_succeeded</code> , <code>step_failed</code>).
<code>status</code> , <code>reason</code> , <code>validation_status</code>	Execution and validation state with optional diagnostic reason.
<code>retry_budget</code> , <code>blocking</code> , <code>needs_user_input</code>	Control flags for replanning and waiting-user decisions.
<code>unmet_preconditions</code> , <code>validation_errors</code>	Structured failure diagnostics for planner handling.
<code>result_summary</code> , <code>result_payload</code>	Backward-compatible summary plus typed skill evidence payload.
<code>step</code>	Step-local trace metadata: id, type, name, on-failure policy, dependencies.

10) Contract H: Planner Dialogue Act (/planner/dialogue_act)

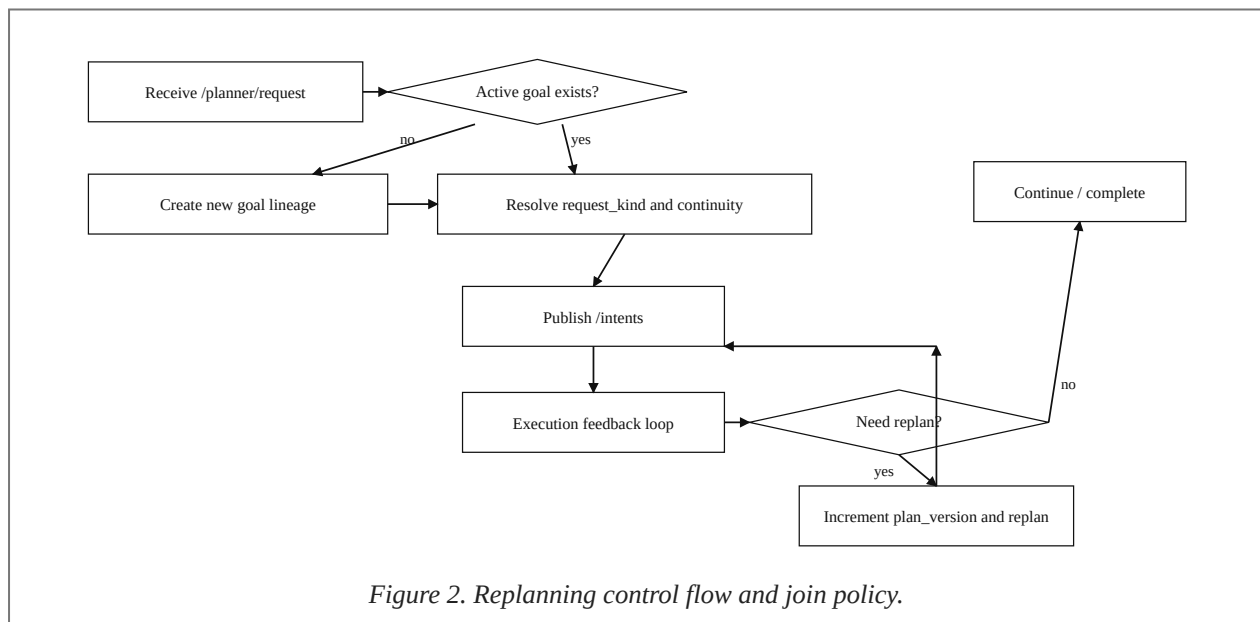
This contract carries planner-originated dialogue intent, decoupled from final wording generation. It supports direct relay or chatbot-mediated realization paths.

```
{
  "goal_id": "goal_turn_123",
  "plan_id": "plan_123",
  "plan_version": 2,
  "act": "ask_clarification",
  "priority": "normal",
  "await_user_response": true,
  "reason": "missing target object",
  "text_hint": "Which person should I look at?",
  "slots_needed": ["target"],
  "context": {
    "scene_targets": ["person"],
    "status": "waiting_user"
  }
}
```

Variable	Description
<code>goal_id</code> , <code>plan_id</code> , <code>plan_version</code>	Lineage binding between dialogue intent and plan state.
<code>act</code>	Dialogue action class. Allowed values: <code>acknowledge</code> , <code>progress_update</code> , <code>ask_clarification</code> , <code>ask_for_help</code> , <code>explain_failure</code> , <code>notify_completion</code> , <code>notify_cancellation</code> .
<code>priority</code>	Relative urgency used for speech priority mapping.
<code>await_user_response</code>	Indicates whether execution should wait for user input.
<code>reason</code> , <code>text_hint</code>	Planner rationale and optional wording hint.
<code>slots_needed</code>	Missing semantic slots required to continue.
<code>context</code>	Compact state payload for dialogue realization and traceability.

11) Replanning and Join Policy

Replanning control is lineage-driven and deterministic. The policy below defines when execution continues, when replanning is triggered, and how join points are resolved safely.



1. Single active-goal queue model.
2. `goal_id` preserves continuity.

3. `plan_id + plan_version` encode lineage.
4. Mid-join uses mapped active `step_id` when present in new plan.
5. Fallback front-join starts from first pending step when mapping fails.
6. Duplicate suppression compares exact tuple `goal_id, plan_id, plan_version`.

References (Companion Project Documents)

- `docs/contracts.md`
- `docs/current_workflow.md`
- `docs/plans/planner_grounding_moe_contract_2026-06-01.md`
- `docs/plans/planner_replan_lineage_adr.md`